Intelligent ML Pipeline for Database Capacity Planning

Author: Ike Ikegwuonu
Publication Year: 2026

Introduction

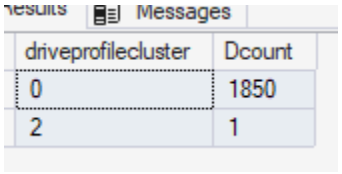
- **Project Statement:** As a Database Administrator (DBA), I frequently receive urgent calls when database servers run out of space. Currently, there is no automated tool in place to monitor drive space usage daily.
- **Modification: P**roactive monitoring relieves the unexpected storage shortages, impacting system performance and availability. By predicting when the drive runs space will allow the DBAs to remain ahead of those issues.
- **Questions:** How can bursty database growth patterns be accurately modeled? Which methods are effective for excluding significant, isolated growth anomalies to prevent them from skewing long-term forecasts? How will the model manage situations where additional space is added to the drive?
- **Overview:** An artificially intelligent ML pipeline that extracts database server information, uses K-Means clustering for anomaly detection, making use of a Random Forest Regressor that forecasts the disk space availability with the following horizons: 30, 60, 90 and 365-day.

Backgrounds

- **Research 1:** Kumar, Jitendra, and Ashutosh Kumar Singh. 'Machine learning for cloud resource provisioning: a survey' Proves that Random Forest models outperform Linear Regression for handling spikes in cloud resource provisioning.

- **Research 2:** Pinheiro, Eduardo, Wolf-Dietrich Weber, and Luiz Andre Barroso. 'Failure trends in a large disk drive population.' Regular metrics are not suitable for disk populations, and it is necessary to detect failure trends pertaining to hardware and anomalies. This can affect space.
- **Good and Bad:** Linear Regression models are handy for date forecasting however they do not do well with high-volume spikes in write requests or data growth. What seems to do well are the ensemble methods like the Random Forest managed complexity well but have a need for clean data and clear from learning garbage noise.
- **Method relation:** The target and solution are to use a hybrid Random Forest for pattern recognition and K-Means filtering for anomaly removal with historical data stored in table for reference.

Methods

- Algorithms: K-Means Clustering assigns sample data into profiles labeled 0 to 3 based on grouping criteria. Queried the anomaly table where the data is put prior to removing it for training. Here is the query:
    - SELECT driveprofilecluster, count(DriveProfileCluster) as Dcount FROM [dba].[dbo].[anomaliesfoundML]|
      group by driveprofilecluster
    - 
      Messages

driveprofilecluster	Dcount
0	1850
2	1

      The data that you see in the screen shot shows that 1850 records were large growth events and the last entry of with the count of 1 in cluster 2 is the test for the anomaly checker and it was removed before the epoch was trained (the data is the only z drive) , so it treats it as a poison pill anomaly and just removes it.

      ```
      INSERT INTO [dba].[dbo].[MonthGrowthSummary]
      ([DTINSERTED], [dbserver], [DriveLetter], [Previous Space], [Current Space], [Growth], [TotalDriveSize], [PercentageFree]
      VALUES
      (GETDATE(), 'SQLSAMPLE-0001', 'Z', 2000000, 100, 1999900, 2097152, 0.001);
      ```

      this shows that it does work.
    - Lets walk through the K-means function

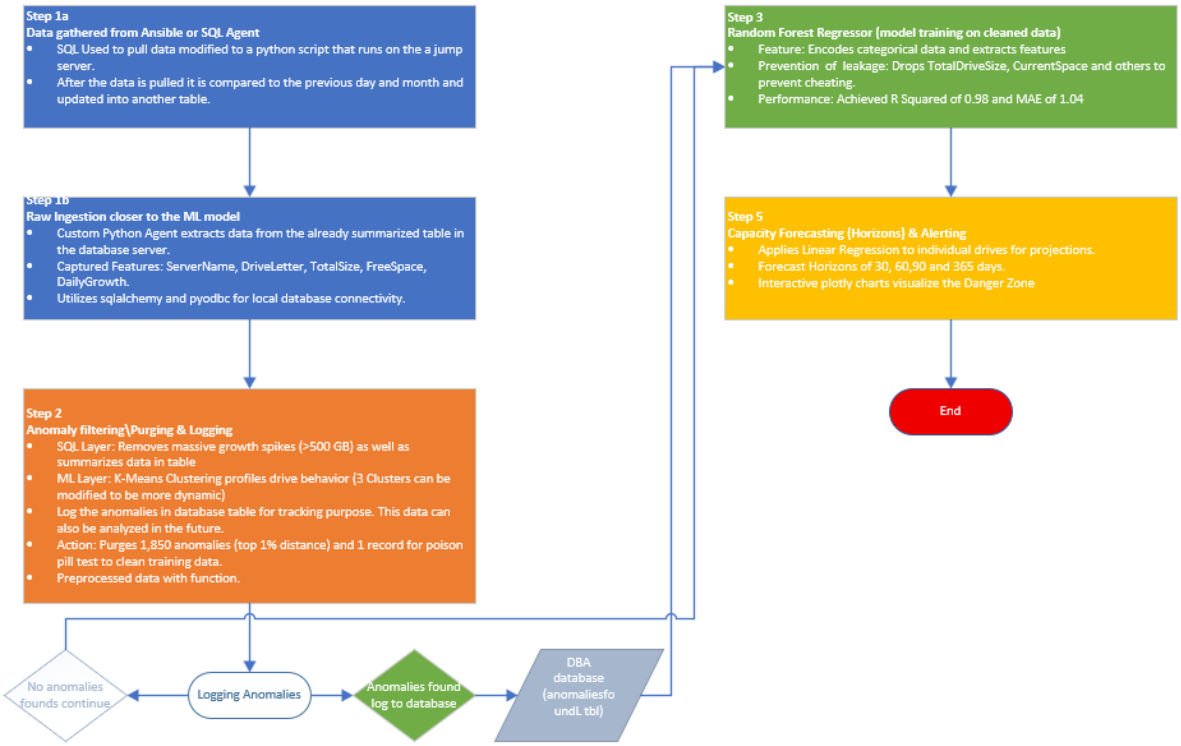
- **Prepares the data**: Most data I pull will not be null so preprocessing semi happens prior to pull the data with SQL into Algorithm. It isolates 3 metrics TotalDriveSize, Growth, and PercentageFree. I will remove any missing data with zeros (non should be returned the preprocess script will ensure that the data is not null). Ensures that massive numbers like terabytes of the TotalDriveSize does completely overpower by using StandardScaler:
- **Drive Clusters**: K-Means has been executed to group the drives into 3 clusters . It assigns the cluster number of (0, 1, 2) to each drive in a new column called DriveProfileCluster.
- **Weird data Score**: K-Means applies by finding the center point (centroid) for each cluster. The function calculates the mathematical distance from each drive to its assigned cluster's center point. The further away a drive is the less it looks like its peers. It saves this information in the DistanceToCentroid.
- **Mark the extremes**: This dynamically assigned all of findings that are in the 99th percentile of all of those distances. The top 1% is marked for review.
- **Poison pill logic**: This was to make sure the anomaly detection works as it should work. It checks the 3 clusters if any has fewer than 5 drives. If found it will assume that the entire group is an anomaly.
- **Flag the extremes**: it creates IsNomaly column. The drive is marked True if it crosses the 99th percentile distance threshold or if it was dumped into one of those tiny cluster with less than 5 drives.

- Last step of K-Means
  - This will actually grab the flagged extremes above and put them into table called anomaliesfoundML_stg.

- **Needs work (future work):** It will then run a merge statement that pulls the data from stg that doesn't match in the adhoc table so only new records will be added.
- **Purge the anomalies after logged:** The last piece of this will just remove all of the anomalies by using the IsAnomaly is false. Then it will remove the DistanceToCentroid and IsAnomaly the prep for training the data.

Framework Figure

Framework Figure: Intelligent ML Pipeline for Database Capacity Planning



(click here)

Data Preprocessing (second round)

- The data before it arrives has gone through some form of preprocessing using SQL language and other methods.

- The timestamps data were converted to numerical date features (year, month, day of the week, day of the year).
- Server names/ drives were all encoded into numerical codes this must happen because it will fail if this is not completed as numpy handles numerical values.

Random Forest Regressor: Used as a primary predictive engine to handle non-linear growth patterns in the data that it receives. Before this process has started we have done several things

- Removal of anomalies so it can have clean data
- Rid of nulls prior to the ML in using SQL code. Will run null cleansing in the ML model as well but it is not needed as the data will not have any nulls.
- **Removal of the following columns**

```
# Prevent Data Leakage: Drop columns that mathmatically equal PercentageFree
columns_to_drop = ["PercentageFree", "DTINSERTED_is_missing", "dbserver_is_missing",
                   "DriveLetter_is_missing", "Current Space", "Previous Space",
                   "Growth", "TotalDriveSize", "LowPercentFree"]
```

  reason behind this is to prevent data leakage. It removes the ability for the model to have the answer key for the data. We are trying to predict PercentageFree, having the CurrentSpace or TotalDriveSIze would allow the model to just perform basic math rather than learning from the patterns.
- **Isolates the target and features:** The dataset is placed in X. This data is where the model will learn from the Y is where the model is trying to predict. It will remove any non-numerical columns as Random Forest model requires numbers.
- **Model gets trained:** It will initialize a RandomForestRegressor, this algorithm will build decision tree and averages the predictions to avoid overfitting, fits it the training data.
- **Performance evaluation:** It will ask the model that has been trained to predict the PercentageFree for the unseen 20% test data. I will grade it using MAE or Mean Absolute Error how far off the data is

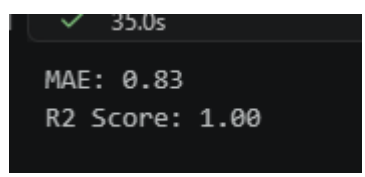
on the average and R-squared to see how well the model explains the variance in the data.

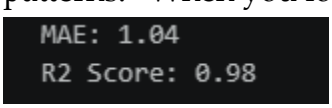
. I get between R2 score of 0.98 to 1.00. This could be because of the scrubbed data. Future features would be to log this as the model ages and maybe come up with some automation that it will fix itself.

Experiments

- **Configured experiments:** With limited access to actual data, I was able to use AI to create server names based on scrubbed data from an actual environment. So, the data you see here is actual data was just scrubbed to ensure I follow some form of policy. But the data generated is from 25 + database servers, with features of Date, FreeSpace, and DailyGrowth.

- **Results:** Reproduced the methods of tree-based models. The Random-squared (R2) is 0.98 on the historical data.

- **The original experiments:** Didn't have K-Means which were going to be future implementations. When I removed K-Means code the R2 Score shoots up to 1.00. So that includes the anomalies etc. So, even though I have more data I have the data that was excluded because of the 1 percent rule that has been turned off and now you have cheating\data leakage as shown. 

This data will lead to overfitting even though those columns were removed you still have the anomalies. The random forest model is likely memorizing the massive growth spikes and throwing off the predictions rather than learning from general patterns. When you look at the original with K-Means you get 

. So, the MAE increases a bit but the test data still is only off 1 percent on the average. The R2 Score also decreased because we got rid of the anomalies. However it still shows 98% of the fluctuations in my drives percentage of free space can be explained by the inputs provided to the model. The other 2 percent is unexplained noise small random changes that don't follow a specific pattern.

- **Discussion:** Kind of to reiterate above the Mean Absolutes Error (MAE) of 1.04 means that predictions were off by 1.04%. This information shown shows that data cleaning and wrangling are more critical to success than the algorithm used.

Conclusions & Future work

- **Conclusion Remarks:** This solution effectively identifies the "Danger Zone" (less than 30 percent free) hitting all the time horizons (30, 60, 90 and 365). This has been proven to work in a live environment currently and the DBAs are using it. The model currently being used doesn't have K-Means yet and will be included in the future release.

- **Future Work**:
    - It could include other metrics like CPU and RAM metrics.
    - Transition from SQL Agent to Python Ansible-based gathering process. This is currently in progress now. The biggest drawback is the method for getting the summarized data from the database servers because of network limitations must think outside of the box to get to it. Maybe having specifications centering around this pulling from a jump server that has access to all the servers it needs to pull the data. This is not meant to just target one technology but for now MSSQL.
    - Implement automated email alerting.
    - Improve the merge statement and only use it on the database server technology.
    - Logging both the Mean Absolute Error (MAE) and the R2 Score to a table with data time to see how the model performs as it receives new data.
    - **Sharing Agreement:** No not now but maybe later in the year, sharing this information; however, this will be under business soon.
    - **Repo can be found here.** https://github.com/tekgeeksolutions/dbgrowthanalyzer

Contributions

- **Clear Explanation:** This project was created by me as a DBA. Designed the original SQL method and basic math to determine the future growth. Decided to add a model and use of K-Means which I have learned recently and it has paid off tremendously. The choice to implement the Random Forest Regressor model was informed by research from a different AI/ML-focused workshop (again it was me researching the best method). It was a crash course, and it had errors I was able to fix with the new knowledge I was able to apply to my current algorithm, which makes use of the current Python libraries.
- Used existing data set and scrubbed the data using AI and loaded MSSQL on my local machine. Took that data using AI to add data. The Scikit-learn, pandas, NumPy libraries I had an idea of what they were before class just reference them in the book. (O'Reilly Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow)
- **What part is my own work?**
    - Custom Data Ingestion Agent: explained above
    - Hybrid Anomaly Filtering (the "poison Pill" logic)
    - Leakage Prevention: explained above
    - Everything that is here in this document was my own work. The original Python script lacked K-Means and Horizons.

References

- Kumar, Jitendra, and Ashutosh Kumar Singh. 'Machine learning for cloud resource provisioning: a survey.' IEEE Access 6 (2018): 3651-3681.
- Pinheiro, Eduardo, Wolf-Dietrich Weber, and Luiz Andre Barroso. 'Failure trends in a large disk drive population.' Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST). Vol. 7. 2007.
- Li, Baichuan, et al. 'Hard drive failure prediction using classification and regression trees.' 2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. IEEE, 2014.